

LEARNING MODULES

- Global Scope variable
- Blocked Scope Variable
- Traditional Loops
 - ✓ While loop
 - ✓ Do While Loop
 - ✓ for Loop
 - ✓ for of
 - ✓ for in
- Functions
 - ✓ Function declaration
 - ✓ Function expression
 - ✓ Arrow function
- Parameters
- Return Values

◆ JavaScript intermediate for Automation

Global Scope Variable:

A global variable is declared outside any function or block. It can be accessed anywhere in the file.

```
let browser = "Chrome"; // Global variable
```

```
function openBrowser() {
```

```
    console.log(browser); // Accessible here
```

```
}
```

```
openBrowser();
```

```
console.log(browser); // Accessible here too
```

◆ JavaScript intermediate for Automation

Blocked Scope Variable:

A block scope variable is declared using `let` or `const` inside `{ }` It is accessible only inside that block

```
if (true) {
```

```
    let status = "Login Successful"; // Block scope console.log(status); // Works here
```

```
}
```

```
console.log(status); // ✗ Error: status is not defined  
{ } can belong to:
```

If-else, Switch case

Loops

Functions

◆ JavaScript intermediate for Automation

var - ✗ Global scope (NOT block)
let - ✓ Block scope
const - ✓ Block scope

```
if (true) {  
    var resultA = "grade A"; let resultB = "grade B" const resultC = "grade C"  
}
```

```
console.log(resultA); // ✓ Works (but risky)  
console.log(resultB); // ✗ not works  
console.log(resultC); // ✗ not works
```

◆ JavaScript intermediate for Automation

TRADITIONAL LOOPS

While Loop:

- Condition checked before execution.
- May run 0 times if condition false.
- Initialization done outside loop.
- Increment/decrement must be written manually.
- Risk of infinite loop if condition not updated.

Example:

```
let i = 0;  
while (i < 3) {  
    console.log(i);  
    i++;  
}
```

◆ JavaScript intermediate for Automation

TRADITIONAL LOOPS

Do While Loop:

- Condition checked after execution.
- Runs minimum one time always.
- Useful when first execution mandatory.
- Less commonly used in real projects.

Example:

```
let i = 5;
```

```
do {  
    console.log(i);  
    i++;  
} while (i < 3);
```

◆ JavaScript intermediate for Automation

MODERN LOOPS

For Loop:

- Initialization runs only once.
- Condition is checked before every iteration.
- Increment/Decrement runs after each iteration.
- Loop stops when condition becomes false.
- Gives full control over loop flow.
- Supports break and continue.
- Best loop when index is required.

Example:

```
for (let i = 0; i < 3; i++) {  
    console.log(i);  
}
```

◆ JavaScript intermediate for Automation

MODERN LOOPS

For of Loop:

- Returns values directly.
- Works on iterable data types.
- Cleaner syntax than for loop.
- Cannot access index directly (but workaround exists).
- Does not work on plain objects.

Example:

```
const fruits = ["Apple", "Mango"];
```

```
for (let fruit of fruits) {  
  console.log(fruit);  
}
```

◆ JavaScript intermediate for Automation

MODERN LOOPS

For in Loop:

- Returns keys, not values.
- Can work on arrays.

Example:

```
const fruits = ["Apple", "Mango"];
```

```
for (let fruit in fruits) {  
  console.log(fruit);  
}
```

◆ JavaScript intermediate for Automation

FUNCTION

Function Declaration:

- Defined using function keyword.
- Must have a function name.
- Hoisted to the top of scope.
- Can be called before declaration.
- Supports this keyword.
- Supports arguments object.
- Used in OOP / prototypes.

Example:

```
function add(a, b) {  
    console.log(a + b);  
}  
add(2, 3)
```

◆ JavaScript intermediate for Automation

FUNCTION

Function Expression:

- Stored in variable.
- Not hoisted like declaration.
- Cannot call before initialization.
- Supports arguments.

Example:

```
const add = function (a, b) {  
    console.log(a + b);  
}  
add(2, 3)
```

◆ JavaScript intermediate for Automation

FUNCTION

Arrow Function:

- Introduced in ES6.
- Stored in variable.
- Not hoisted.
- Does NOT have its own this.
- Best for short functions.

Example:

```
const add = (a, b) => console.log(a + b);
```

```
add();
```

◆ JavaScript intermediate for Automation

FUNCTION

Parameters:

- $a, b \rightarrow$ Parameters
- $2, 3 \rightarrow$ Argument

Example:

```
function add(a, b) {  
    console.log(a + b);  
}
```

```
add(2, 3);
```

◆ JavaScript intermediate for Automation

FUNCTION

Return Values:

return sends value back from function to caller.

Example:

```
function add(a, b) {  
    return a + b;  
}  
const result = add(2, 3);  
console.log(result);
```